

PROMO X- SEMESTRE Y- FORMATION

Nom de l'Enseignant :
Haby DIALLO

Cours :
Administration de base de données

Séquence 3





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

Objectifs spécifiques :

A la fin de ce chapitre, l'étudiant doit être capable de:

- Définir une transaction
- Connaître les commandes : COMMIT et ROLLBACK
- Connaître les propriétés d'une transaction : ACID
- Savoir utiliser des transactions (accès concurrents)
- Résoudre le problème de verrouillage
- Résoudre des cas d'inter blocage





Titre: CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

10 ans
2013 - 2023



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Définition:** Une transaction est une séquence d'opération de lecture ou de mise à jour sur une base de données, se terminant par l'une des deux instructions suivantes: commit ou roll back
- **Une transaction doit être ACID**



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **ACID: Atomicité, Cohérence, Isolation, Durabilité**
- **Atomicité:** Une transaction doit être soit entièrement réalisée, soit annulée. Aucun état intermédiaire n'est accepté. Une transaction est considérée comme une unité logique de travail unique et indivisible.

- Exemple:

Deux personnes qui veulent manipuler le même compte solde 100000 francs :

P1 : L'une veut faire un retrait de 100000 francs

P2 : L'autre veut faire un versement de 250000 francs

P1 : Thiès

- Solde = Lire(compte) solde = 100000
- Solde = Solde - 100000 Solde = 0
- Ecrire (solde) Solde = 0

P2 : Kaolack

- Solde = Lire(compte) solde = 100000
- Solde : Solde + 250000 Solde = 100000 + 250000 = 350000
- Ecrire (solde) Solde = 350000



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **ACID: Atomicité, Cohérence, Isolation, Durabilité(suite)**
- **Cohérence** : Un état de la base de données est cohérent si toutes les contraintes d'intégrité des données sont satisfaites. Une transaction doit transformer une base de données cohérente en une autre base aussi cohérente.
- Exemple: Imaginons maintenant que, pendant le déroulement d'une des réservations, une défaillance ait lieu et que la réservation du trajet Madrid-Paris n'ait pas été enregistrée. Dans ce cas, le client aura son billet mais sa réservation n'apparaîtra pas dans la base de données de l'entreprise ferroviaire.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **ACID: Atomicité, Cohérence, Isolation, Durabilité(suite)**
- **Isolation** : Une transaction ne peut pas montrer son effet aux autres transactions avant sa validation.
- Durant l'exécution de la transaction, les données ne peuvent pas être utilisées par une seconde transaction jusqu'à ce que la première soit terminée.
- On a besoin de cette propriété pour éviter le problème de l'abandon en cascade.
- Par exemple, si une seconde transaction fait l'émission d'un billet, elle ne peut pas voir les états intermédiaires de la transaction qui fait les trois réservations.
- Si on fait l'abandon des réservations, on doit faire aussi l'abandon de l'émission et ça n'est pas possible parce que le système ne peut pas défaire le billet.
- Par nature, les systèmes de bases de données mono-utilisateur assurent automatiquement l'isolation de la base de données car une seule transaction est exécutée à la fois.
- L'atomicité et la durabilité doivent néanmoins être garanties..



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **ACID: Atomicité, Cohérence, Isolation, Durabilité(suite)**
- **Durabilité** : Ceci indique la permanence de l'état cohérent de la base de données.
- Lorsqu'une transaction est validée, la base reste dans un état cohérent et cet état ne peut pas être perdu même en cas de défaillance du système.
- Une fois que le commit est exécuté, aucune défaillance ne peut changer les résultats ou les perdre.
- Imaginons ce qui se passerait si après que le client ait réservé et payé, la réservation était annulée par erreur.
- Les systèmes de bases de données avec l'accès de plusieurs utilisateurs font l'objet de transactions concurrentes.
- Le système de gestion de la base de données doit effectuer des contrôles pour garantir la cohérence de la base de données.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Opérations élémentaires**
- Les utilisateurs ont la possibilité de manipuler les données de la base suivant des opérations élémentaires.
- Une transaction a une marque de début « **Begin of transaction** » et une marque de fin « **end of transaction** ». Entre ces deux marques il y a les opérations élémentaires.
- Dans les bases de données, et bien sûr dans le standard SQL, les requêtes qu'un utilisateur fait se traduisent par les opérations élémentaires suivantes sur les données:
 - **READ** : Lecture. On lit la valeur des données d'un objet.
 - **WRITE** : Ecriture. On écrit la nouvelle valeur des données d'un objet.
 - **COMMIT** : Validation. Après avoir réalisé certaines opérations (READ, WRITE), le processus qui a commencé indique qu'il désire contrôler si tout s'est bien déroulé.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Opérations élémentaires (suite)**
- Si tous les processus sont d'accord, les changements des opérations précédentes sont enregistrés de manière permanente.
- Cette opération termine automatiquement une transaction. Sinon, les processus abandonnent la transaction (**ABORT**).
- **ABORT** : Abandon. Tous les changements des opérations précédentes sont annulés et la base de données revient dans l'état cohérent qui était le sien avant le début de la transaction.
- Lorsqu'une transaction est lancée par un utilisateur ou un programme d'application, ses opérations s'exécutent avec succès jusqu'à ce qu'une opération **COMMIT** ou **ABORT** soit rencontrée, ceci mettant fin à la transaction.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Les transactions concurrentes**
- Il y a transactions concurrentes dès que plusieurs utilisateurs veulent accéder à une même donnée.
- **Les systèmes multitâches**
 - Un critère pour classer les systèmes de base de données est le nombre d'utilisateurs qui peuvent utiliser le système en concurrence, c'est à dire en même temps.
 - Un système est multiutilisateur si plusieurs utilisateurs peuvent y accéder en concurrence.
 - Les utilisateurs peuvent utiliser des systèmes en concurrence parce que les systèmes informatiques actuels ont la possibilité de travailler en multitâches.
 - Si une seule CPU est présente, elle peut exécuter au plus une tâche (programme ou transaction) à la fois.
 - Le travail en multitâche en vue d'optimisation du temps CPU est géré par l'ordonnanceur.



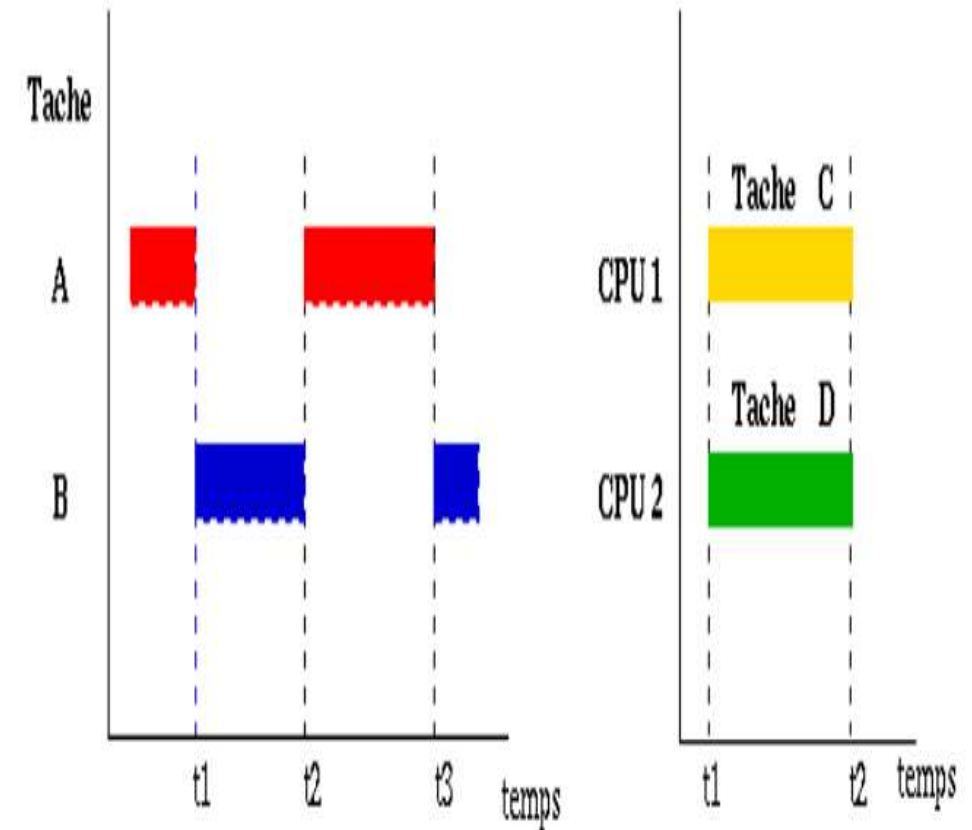
CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Les transactions concurrentes(suite)**
- **Les systèmes multitâches**
 - Suivant un ordre de priorité (s'il y en a un) ou suivant un ordre arbitrairement défini, l'ordonnanceur va attribuer le processeur à chaque tâche à tour de rôle :
 - il exécute quelques commandes d'une tâche, suspend ensuite cette tâche pour exécuter quelques commandes de la tâche suivante et ainsi de suite.
 - Chaque tâche est reprise là où elle avait été suspendue.
 - Chaque utilisateur a ainsi l'impression d'être seul avec sa machine.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

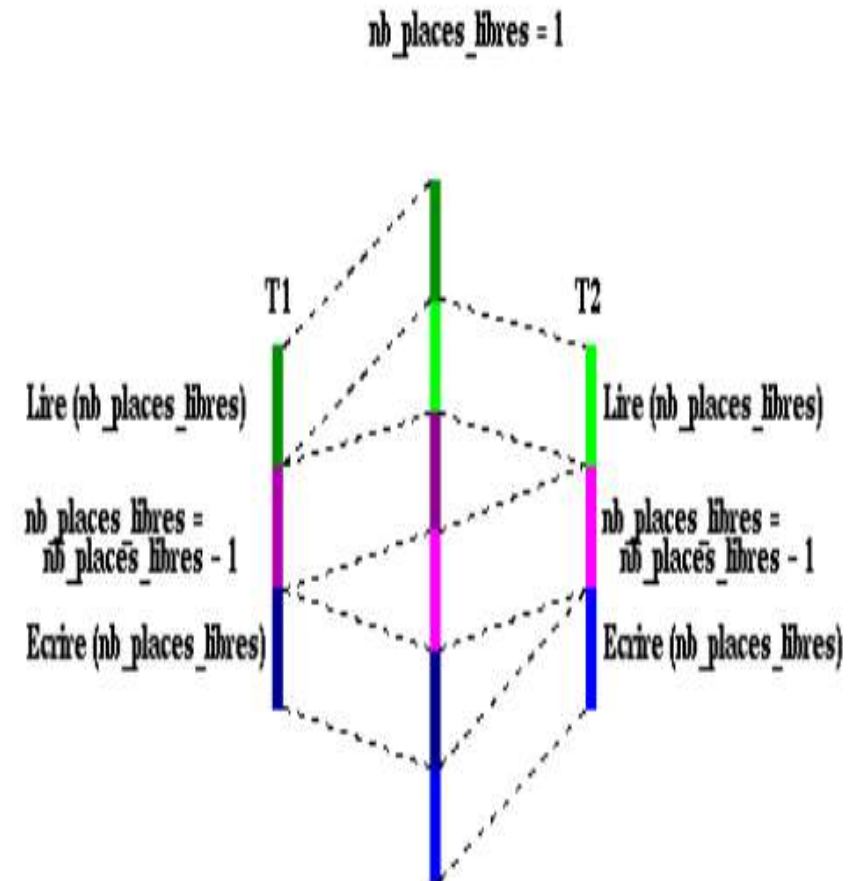
- **Les transactions concurrentes(suite)**
- **Les systèmes multitâches**
 - Comme l'illustre la figure suivante, l'exécution concurrente des programmes A et B se fait de manière imbriquée.
 - S'il y a plusieurs CPUs, l'exécution simultanée de plusieurs tâches est possible (comme illustré par les tâches C et D), conduisant à une concurrence simultanée plutôt qu'imbriquée.





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Problèmes posés par les transactions concurrentes**
- Considérons deux transactions faisant une réservation d'une place d'avion, alors qu'il n'y a plus qu'une place de disponible.
- Les deux transactions pourront néanmoins effectuer les deux réservations, conduisant à une incohérence de la base de données.
- Plusieurs problèmes peuvent ainsi se poser lorsque des transactions concurrentes s'exécutent de manière non contrôlée





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Problèmes posés par les transactions concurrentes(suite)**
- **Perte de mise à jour**
- La perte de la mise à jour d'une donnée peut avoir lieu lorsque deux transactions qui accèdent aux mêmes données ont leurs opérations imbriquées, si bien qu'une valeur d'une donnée devient incorrecte.
- Ainsi sur la figure suivante, la valeur finale de X est incorrecte, car T2 lit la valeur de X avant que T1 la change dans la base de données.
- Si $X = 80$ au début, T1 retranche 5 et T2 ajoute 4, la valeur finale devrait être 79.
- Mais elle sera de 84, car la mise à jour $X-5$ de T1 est perdue.
- L'anomalie de perte de mise à jour résulte de la dépendance Ecriture-Ecriture entre actions de plusieurs transactions interférentes.

T1	T2
Lire (X) $X = X - 5$	Lire (X) $X = X + 4$
Ecrire (X)	Ecrire (X)



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

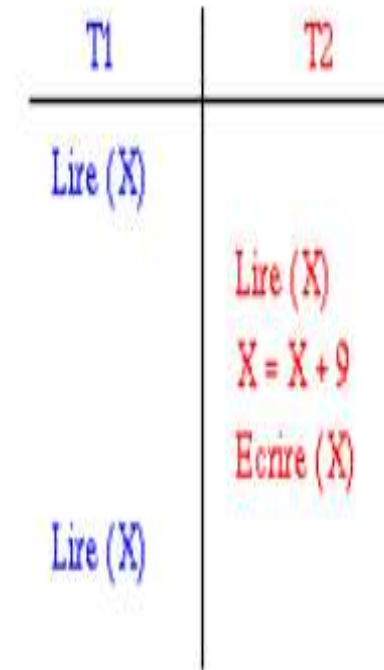
- **Problèmes posés par les transactions concurrentes(suite)**
- **Lecture impropre (Dirty read):**
 - Une lecture d'une valeur temporaire de donnée peut conduire à une valeur fausse.
 - Ceci a lieu lorsqu'une transaction met à jour une donnée de la base et que cette transaction échoue ou est annulée.
 - La valeur mise à jour est lue par une seconde transaction avant que la donnée ait retrouvé sa valeur initiale conduisant à un "dirty read".
 - Dans la figure suivante, T1 met à jour la donnée X et ensuite échoue et est annulée : X retrouve sa valeur initiale (80). T2 a lu entre-temps la valeur temporaire de X (75) qui ne sera pas enregistrée à cause de l'échec de T1. La valeur de X lue par T2 est donc fausse.
 - L'anomalie de lecture impropre résulte de la dépendance Écriture-Lecture entre actions de différentes transactions

T1	T2
Lire (X)	
$X = X - 5$	
Ecrire (X)	
	Lire (X)
	$X = X + 4$
	Ecrire (X)
ABORT	



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Problèmes posés par les transactions concurrentes(suite)**
- **Lecture non reproductible**
- Dans le corps de la même transaction, la relecture d'une donnée peut produire un résultat différent du fait de la modification de la donnée par une autre transaction entre les deux lectures successives.
- Sur la figure suivante, T1 relit la valeur de X après que T2 la change dans la base de données.
- Si $X = 80$ au début, T1 lue 80 et T2 aussi. T2 ajoute 9 et quand T1 relue la valeur de X, elle est 71. Mais elle devrait être 80 car T1 n'a pas changé la valeur de X.
- L'anomalie de lecture non reproductible est un cas particulier de lecture incohérente due à l'observation d'états momentanément incohérents.
- Cette anomalie qui résulte de la dépendance Lecture-Ecriture entre actions correspond à une destruction de l'intégrité interne des données.





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

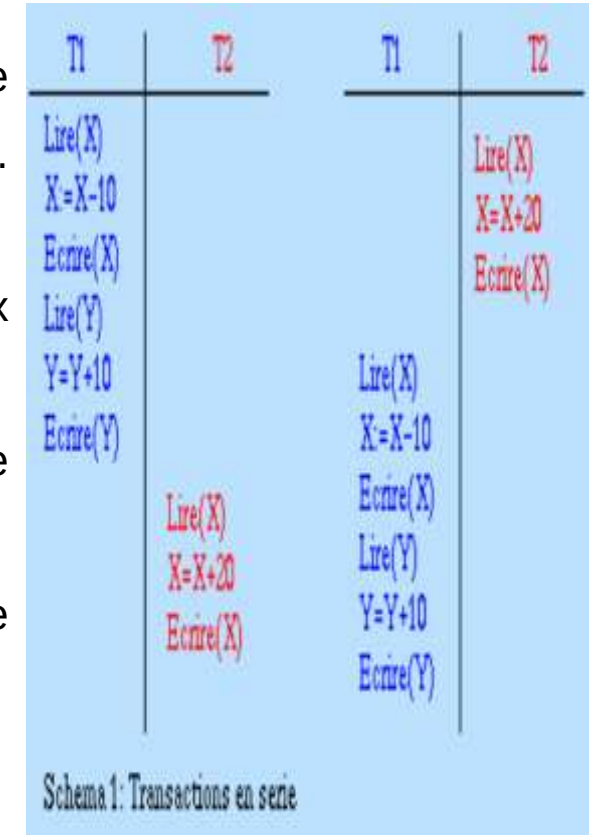
- **Problèmes posés par les transactions concurrentes(suite)**
- **Lecture non reproductible**
- Nous pouvons noter qu'il n'y a pas d'anomalies correspondant à la dépendance Lecture- Lecture car la lecture est commutative.
- Par ailleurs, il peut arriver que du fait du travail en multitâches, le processeur n'exécute pas les transactions dans l'ordre de leur demande par les utilisateurs : La base de données peut alors contenir des données erronées.
- Il apparaît ainsi que le principe de base de gestion des transactions concurrentes est la notion de sérialisabilité.

T1	T2
Lire (X)	Lire (X)
	$X = X + 9$
	Ecrire (X)
Lire (X)	



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

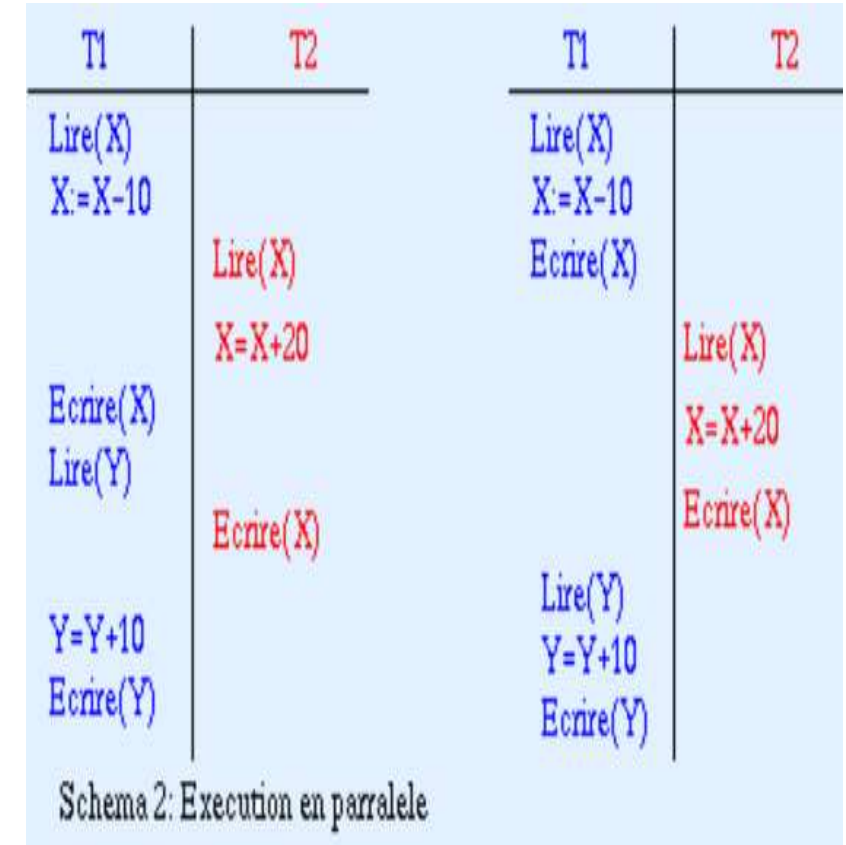
- **Principe de base du contrôle de concurrence**
- **Approche**
- Supposons maintenant que deux agents de deux agences de voyage différentes veulent effectuer chacun une transaction sur une même base de données. Supposons de plus que ces transactions sont exécutées séquentiellement
- Pour l'exécution, il existe deux façons d'ordonner les opérations constituant les deux transactions :
 1. On exécute toutes les opérations de T1 consécutivement, puis on fait de même avec les opérations de T2.
 2. On exécute toutes les opérations de T2 consécutivement, puis on fait de même avec T1.
- Le schéma suivant montre la différence entre les 2 exécutions:
- Ces deux exécutions en série, qui ne diffèrent que par l'ordre des transactions, laissent la base par définition cohérente, dans un état stable.





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Approche**
- Supposons maintenant que les transactions ne sont plus exécutées les unes après les autres, mais sont exécutées en multitâche.
- On se rend alors compte que plus de deux alternatives sont possibles. Un exemple en est illustré ci-contre:
- Un aspect important du contrôle de concurrence est la théorie de la sérialisabilité: Cette théorie vise à déterminer quelles exécutions sont « correctes » ou pas et permettent de développer des techniques qui forcent la sérialisabilité d'exécutions.
- Avant de définir clairement la notion de sérialisabilité, il est nécessaire d'introduire la notion d'exécutions équivalentes.





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Notion d'équivalence d'exécution
- Première définition
- On dit que deux exécutions sont équivalentes si elles produisent le même résultat: même état final de la base de données.
- Cependant, cette définition est ambiguë puisqu'il peut arriver que deux exécutions donnent accidentellement le même résultat pour une donnée particulière sans pour autant être équivalentes.
- L'exemple qui suit illustre bien le problème.
- Nous allons donc changer la définition de l'équivalence. Pour ce faire, définissons la notion d'opérations en conflit.

Soit les deux exécutions suivantes:

S1	S2
Lire(X)	Lire(X)
$X = X * X;$	$X = X - 3;$
$X = X - 3;$	$X = X * X;$
Ecrire(X)	Ecrire(X)

Pour $X=2$, on obtient le même résultat dans les deux exécutions S1 et S2.

Mais en général, le résultat est différent.

Ces 2 exécutions ne sont pas équivalentes.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Des opérations conflictuelles**
- Deux opérations sont conflictuelles lorsqu'elles proviennent de deux transactions différentes, qu'elles veulent accéder à un même objet de la base de données et que l'une de ces opérations est une opération d'écriture.
- C'est en effet dans ce cas, qu'apparaissent des conflits. En voici un exemple:
- L'opérateur 1 veut lire une donnée O1 et L'opérateur 2 veut changer la donnée O1 et donc avoir un accès en écriture de O1.
- Si les transactions sont exécutées en parallèle, il va falloir déterminer qui de l'opérateur 1 ou 2 aura la main en premier sur l'objet O1: L'ordre de ces 2 transactions détermine le résultat.
- On peut en déduire une deuxième définition de l'équivalence entre exécutions:
- **Deuxième définition de l'équivalence de deux exécutions.**
- On dira que deux exécutions sont équivalentes si l'ordre d'exécution de toutes opérations conflictuelles est le même dans les deux exécutions.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- La sérialisabilité
- **Définition:** une exécution en parallèle de n transactions est sérialisable si et seulement si elle est équivalente à une exécution en série (les unes après les autres) de ces mêmes n transactions.
- Exemple de sérialibilité

Soit 3 transactions sur 2 objets 01 et 02 :

T1
LECT(01)
LECT(02)

T2
LECT(01)
ECRIT(01)

T3
ECRIT(02)
LECT(02)



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- La sérialisabilité

On effectue une execution en parralele S de ces 3 transactions :

T1	LECT(O1)
T2	LECT(O1)
T3	ECRIT(O2)*
T1	LECT(O2) *
T2	ECRIT(O1)
T3	LECT(O2)

On peut observer les transactions en conflits pour chaque objet :

Pour O1: Pas de transactions en conflits.

Pour O2: Les operations d'écriture de T3 et de lecture de T1
sont en conflits. On les marque d'un asterix.

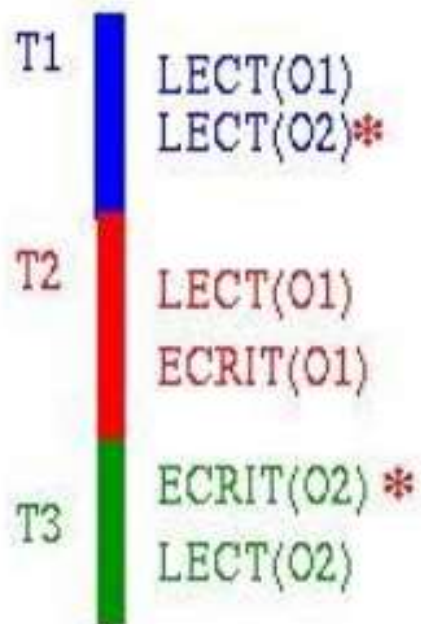
L'ordre dans lequel vont s'effectuer
ces operations T1 et T3 est primordial pour la recherche d'une
execution serie equivalente a cette execution en parralele.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- La sérialisabilité

Soit l'exécution en serie S1 de ces 3 transactions dans l'ordre T1 T2 T3 :



Remarquons d'abord que l'ordre d'exécution de S1 (T1,T2,T3) correspond à l'ordre pris pour ordonnancer nos transactions dans l'exécution en parallèle S précédente.

Voyons l'ordre dans lequel sont exécutées les 2 opérations conflictuelles de T1 et de T3:

Ici, Les 2 opérations conflictuelles ne sont pas exécutées dans le même ordre que précédemment.

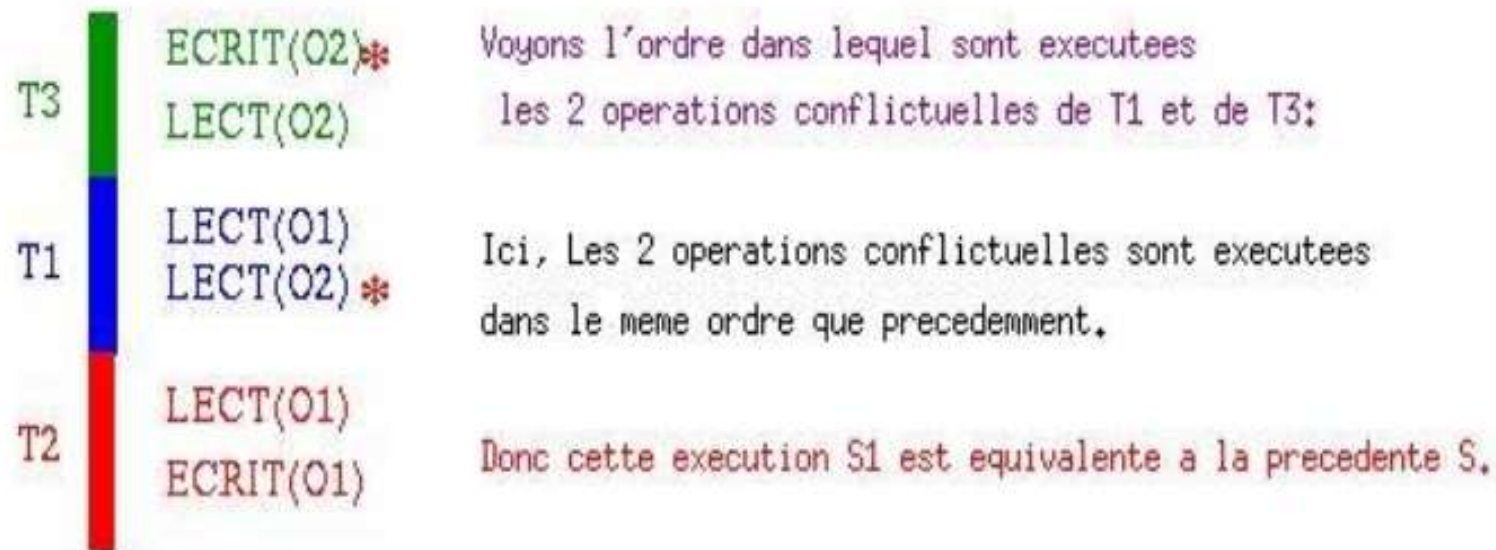
Donc cette exécution S1 n'est pas équivalente à la précédente S.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- La sérialisabilité

Soit l'exécution en série S2 de ces 3 transactions dans l'ordre T3 T1 T2 :





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- La sérialisabilité

Soit l'exécution en serie S2 de ces 3 transactions dans l'ordre T3 T1 T2 :

T3	ECRIT(O2)* LECT(O2)	Voyons l'ordre dans lequel sont executees les 2 operations conflictuelles de T1 et de T3:
T1	LECT(O1) LECT(O2)*	Ici, Les 2 operations conflictuelles sont executees dans le meme ordre que precedemment.
T2	LECT(O1) ECRIT(O1)	Donc cette execution S1 est equivalente a la precedente S.

Cet exemple illustre bien la notion d'equivalence d'execution
et la notion de serialisabilite

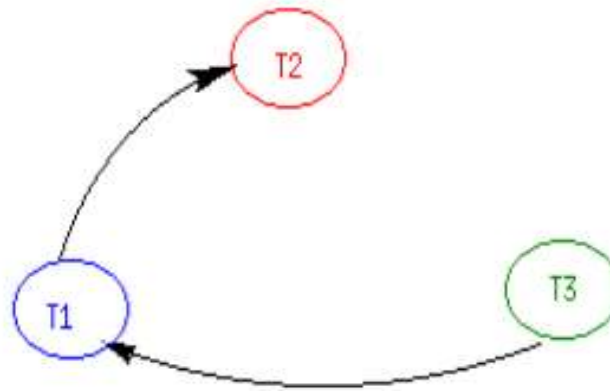
On remarquera que l'ordre dans lequel sont executees les transactions
dans l'execution en parrallele (S) est le meme que l'execution en serie S2
et que, neanmoins, ces deux executions ne sont pas equivalentes.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Graphe de sérialisabilité

T1	LECT(O1)	
T2	LECT(O1)	
T3	ECRIT(O2)	*
T1	LECT(O2)	*
T2	ECRIT(O1)	
T3	LECT(O2)	



Pas de cycle dans le graphe de serialisabilite de S:

On voit donc que S est serialisable

et que une execution serie equivalente de S est T3-T1-T2 (S2),

Comme nous l'avions vu precedemment.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Algorithme et graphe de sérialisabilité**
- Il existe un algorithme qui permet de déterminer la sérialisabilité d'une exécution.
- Cet algorithme met en œuvre des relations de précédence entre les transactions: Ces relations déterminent par rapport à une exécution en parallèle choisie l'ordre dans lequel doivent s'exécuter les transactions.
- Une fois toutes les relations de précédence établies, on peut générer un graphe dit de sérialisabilité.
- La théorie dit alors que l'on a une exécution non sérialisable si et seulement si le graphe de sérialisabilité de cette exécution possède un cycle.
- Autrement dit, une exécution est sérialisable si et seulement si son graphe de sérialisabilité ne présente aucun cycle



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Algorithme et graphe de sérialisabilité**
 1. Chaque transaction est un nœud du graphe;
 2. Pour un objet O :
 - ✓ Si une transaction T1 fait une écriture de l'objet O avant qu'une transaction T2 ne fasse une opération sur cet objet alors, dans le graphe, T1 précède T2 :
 - ✓ Cette relation de précédence est concrétisée par une branche orientée de T1 vers T2.
 3. Pour un objet O :
 - ✓ Si une transaction T1 fait une lecture sur l'objet O avant qu'une transaction T2 ne fasse une opération d'écriture sur cet objet, alors, dans le graphe, T1 précède T2:
 4. L'exécution est sérialisable si et seulement si le graphe ne présente aucun cycle.

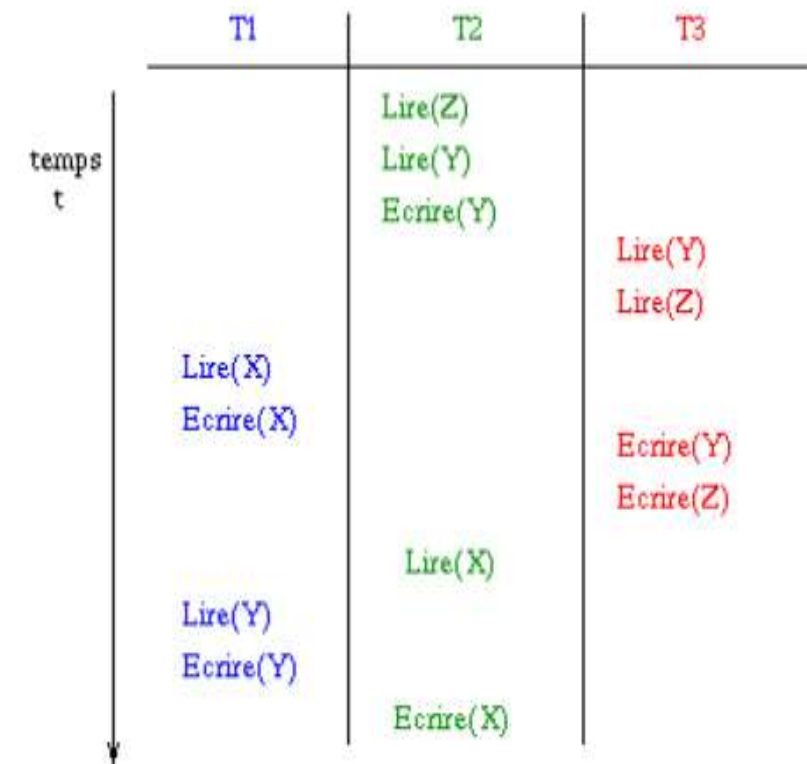


CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Réalisation d'un graphe de serialisabilite
- Comme vu précédemment, le graphe de serialisabilite traduit les relations de précédence entre les transactions impliquées dans une exécution.
- Ces relations dépendent fortement de l'ordre d'exécution des opérations élémentaires des transactions.
- Nous allons voir sur trois exemples, comment se traduit une exécution en termes de relations de précédence.
- Dans un premier temps, nous établissons pour chaque objet la liste des transactions qui accèdent a l'objet par ordre chronologique. Dans un second temps, nous appliquons l'algorithme de serialisabilite.

Exemple 1.

Schema 1: Transactions en serie





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Realisation d'un graphe de serialisabilite

Pour la ressource X: T1 : Lire(X), Ecrire(X)

T2 Lire(X), Ecrire(X)

T1 précède T2

Pour la ressource Y: T2 : lire(Y) Ecrire (Y)

T3: lire(Y), Ecrire(Y)

T1 : LireY), Ecrire(Y)

T2 précède T 3 qui précède T1 donc T2 précède T1

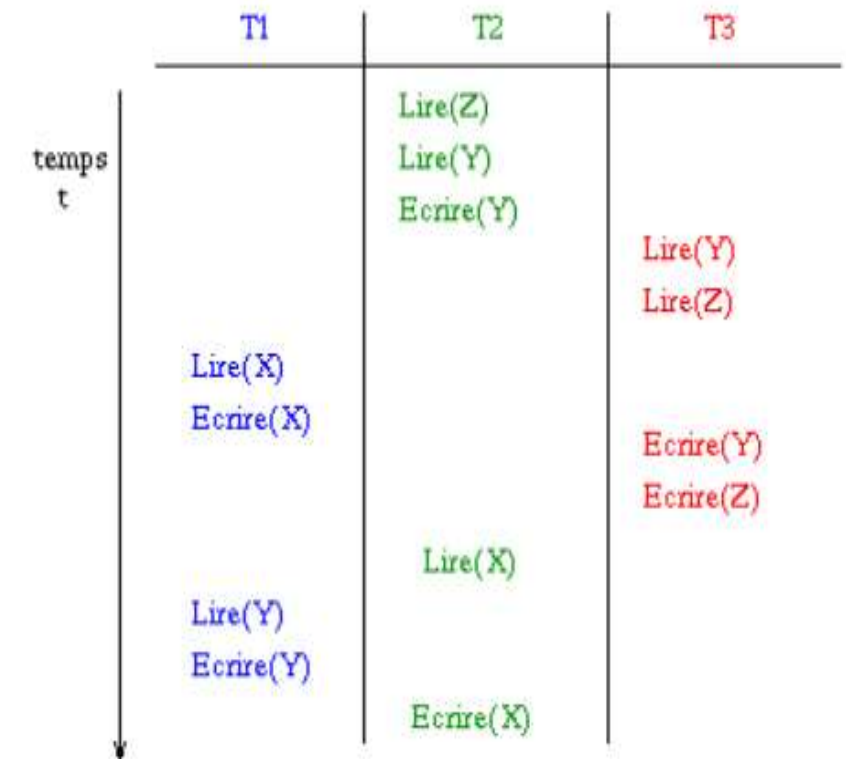
Pour la ressource Z : T2 : lire(Z)

T3: Lire(Z), Ecrire(Z)

T2 précède T3

Exemple 1.

Schema 1: Transactions en serie





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Realisation d'un graphe de serialisabilite

Pour L'objet X:

Lecture de T1, Écriture de T1, Lecture de T2, Ecriture de T2. Relation de precedence: T1 precede T2.

Pour l'objet Y:

Lecture de T2, Ecriture de T2, Lecture de T3, Ecriture de T3, Lecture de T1, Ecriture de T1.

Relation de précédence: T2 écrit avant T3 donc T2 précède T3; T2 écrit avant T1 donc T2 précède T1;

T3 écrit avant T1 donc T3 précède T1.

Pour l'objet Z:

Lecture de T2, Lecture de T3, Ecriture de T3.

Relation de précédence: Le fait que T2 lise avant que T3 ne lise ne provoque pas de conflit: les deux premières opérations de la liste précédente ne signifie pas que T2 précède T3.

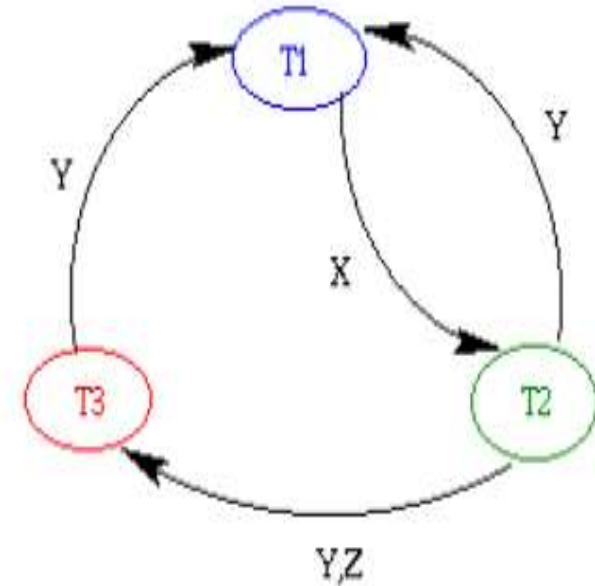
Cependant, la précédence doit être exprimée ici, parce que T3 fait par la suite une opération d'écriture sur Z.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
 - Réalisation d'un graphe de serialisabilite
 - Construction du graphe de précédence
1. La construction en est simple, une fois les relations de précédence étudiées.
 2. Toutes les transactions sont représentées par des noeuds
 3. On indique que T1 précède T2 en dessinant une flèche de vers T2.
 4. Pour plus de clarté, on peut mettre sous chaque flèche, l'objet a partir duquel a été déduite la relation représentée la flèche.

Le graphe de précédence ressemble alors a ce qui suit:



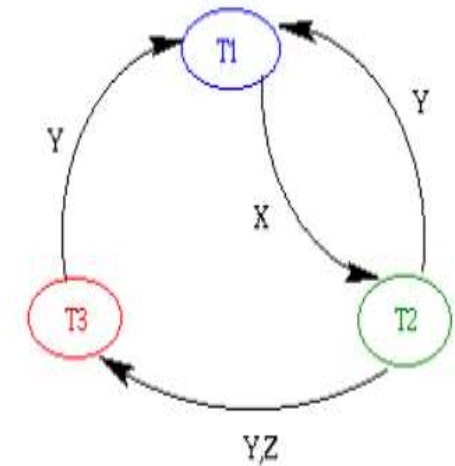
$X(T1 \rightarrow T2); Y(T2 \rightarrow T1)$

$X(T1 \rightarrow T2); YZ(T2 \rightarrow T3); Y(T1 \rightarrow T3);$



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Réalisation d'un graphe de serialisabilité
 - Conclusion: On ne peut pas trouver d'exécution de T1, T2, T3 en série, car il y a présence d'un cycle (deux cycles) dans le graphe.
 - Ceci signifie que si l'on effectue dans cet ordre, en multitâche, les transactions T1, T2, T3, on n'obtiendra pas sauf cas particulier, le résultat voulu.
 - Ce résultat devrait être le même que le résultat de trois transformations finies, atomiques de la base de données exécutées dans un ordre quelconque: Qu'on fasse T1, T2, T3 ou T2, T3, T1, on obtiendra le même état final de la base de données.



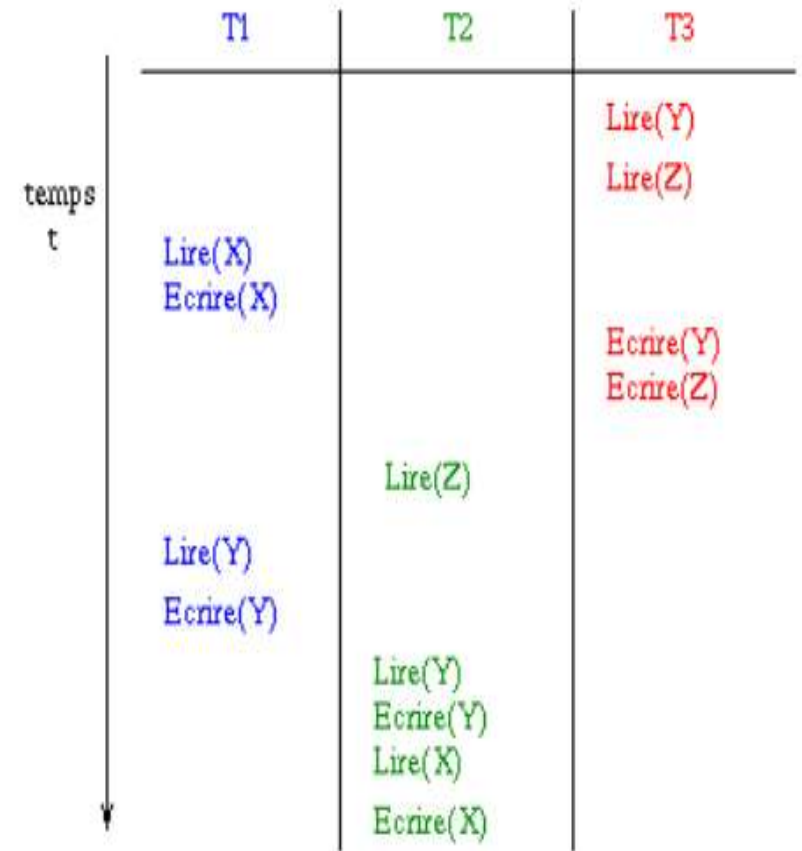
$X(T1 \rightarrow T2); Y(T2 \rightarrow T1)$

$X(T1 \rightarrow T2); YZ(T2 \rightarrow T3); Y(T1 \rightarrow T3);$



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Réalisation d'un graphe de serialisabilite
 - Exemple 2.
 - Nous prenons les mêmes transactions mais exécutées en parallèle dans un ordre tout à fait différent:
 - Comme précédemment, dans un premier temps, nous établissons pour chaque objet la liste des transactions qui accèdent a l'objet par ordre chronologique.
 - Et dans un second temps, nous appliquerons l'algorithme de serialisabilite.





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Réalisation d'un graphe de serialisabilité
 - Exemple 2.

Pour la ressources X: T1: lire(X), Ecrire (X)

T2: Lire(X), Ecrire (X)

T1 précède T2 sur X

Pour la ressource Y: T3 : lire (Y), Ecrire(Y)

T1: Lire(Y), Ecrire(Y)

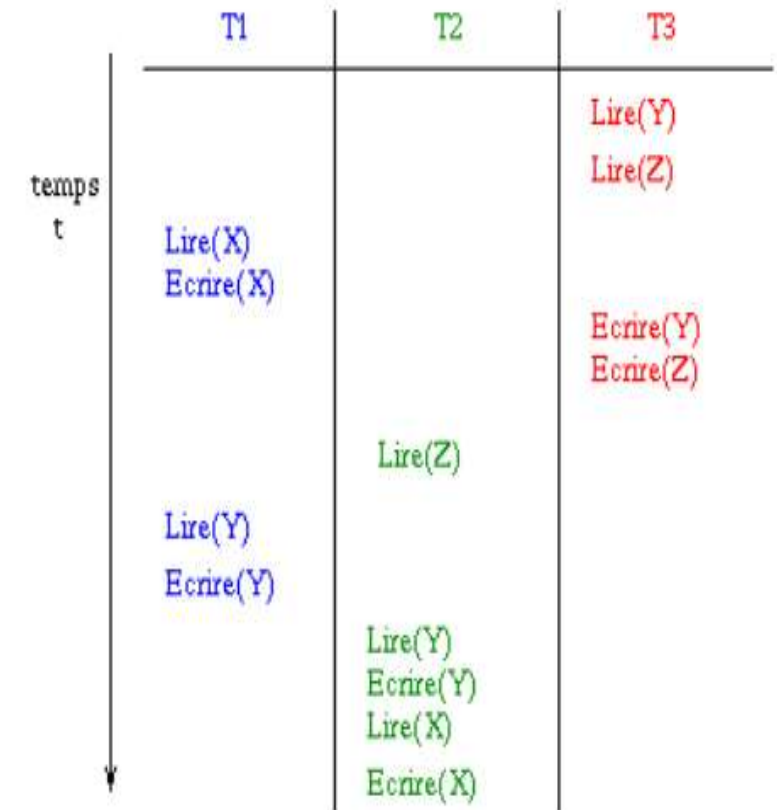
T2 : Lire(Y), Ecrire(Y)

T3 qui précède T1 qui Précède T2 donc T3 Précède T2

Pour la ressource Z: T3 Lire(Z) , Ecrire (Z)

T2: lire(Z)

T3 qui précède T2





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Principe de base du contrôle de concurrence(suite)
- Réalisation d'un graphe de serialisabilité
- Exemple 2.

Pour L'objet X:

Lecture de T1, Écriture de T1, Lecture de T2, Écriture de T2.

Relations de précédence: On détermine sans difficulté que T1 précède T2.

Pour l'objet Y:

Lecture de T3, Écriture de T3, Lecture de T1, Écriture de T1, Lecture de T2, Écriture de T2.

Relations de précédence: T3 écrit avant T1 donc T3 précède T1; T3 écrit avant T2 donc T3 précède T2;

T1 écrit avant T2 donc T1 précède T2.

Pour l'objet Z:

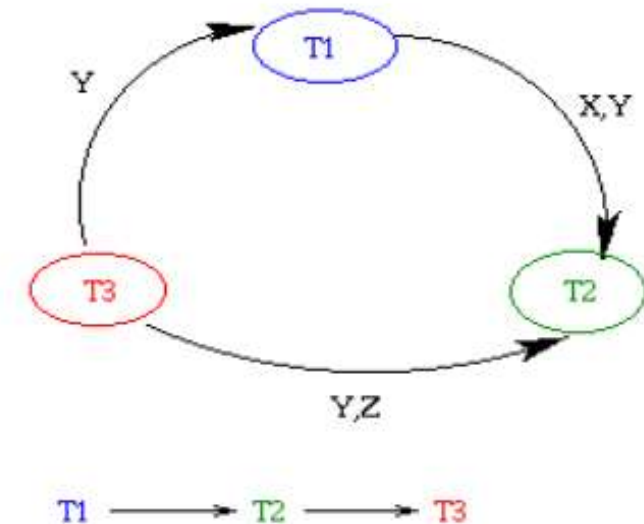
Lecture de T3, Écriture de T3, Lecture de T2.

Relations de précédence: T3 écrit avant T2 donc T3 précède T2.

CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Réalisation d'un graphe de serialisabilite**
- **Conclusion:** Il n'y a pas de cycle dans notre graphe de serialisabilite. On peut donc donner de cette exécution une exécution en série équivalente:
- Cela signifie que l'état de la base de données (quelles que soient les valeurs des données) sera le même après l'exécution en parallèle des 3 exécutions dans l'ordre donné qu'après l'exécution série équivalente T3, T1, T2.

Le graphe de précédence ressemble alors a ce qui suit:





CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Principe de base du contrôle de concurrence(suite)**
- **Utilisation de la serialisabilite**
- Il est clair que l'exécution de transactions en série n'est absolument pas efficace en terme d'optimisation de temps CPU, temps de traitement de l'information pour le processeur.
- En pratique, il est quasiment impossible de tester une exécution en termes de sérialisabilité : en effet, l'ordonnancement des opérations des différentes transactions impliquées est typiquement traité par le processeur : Il est très difficile de le connaître en général, à priori. Il dépend de nombreux facteurs: les priorités des transactions, le temps de soumission d'une transaction...
- Déterminer l'ordre d'exécution des opérations des différentes transactions est en pratique impossible à faire. Il devient alors impossible de créer le graphe de sérialisabilité et donc de définir si oui ou non, cette exécution est sérialisable.
- résout alors le problème du contrôle de concurrence en utilisant des méthodes qui assure qu'une exécution sera sérialisable. Ces méthodes sont en fait des ensembles de protocoles et de règles qui sont imposées/appliquées par le système de gestion de base de données.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence**
- **Le verrouillage**
- Le contrôle de concurrence consiste à coordonner l'exécution simultanée de transactions dans un système multitâches de gestion de base de données. L'objectif est d'assurer la sérialisabilité des transactions.
- Un bon moyen d'assurer la sérialisabilité est d'imposer que les accès aux données de la base se fassent de façon mutuellement exclusive.
- Un verrou garantit un usage exclusif d'une donnée à une transaction. Un verrou sert à interdire l'accès à une donnée qui est en cours d'utilisation par une autre transaction. Si une transaction T1 pose un verrou sur l'accès d'une donnée, une transaction T2 n'aura pas accès à cette donnée.
- Le verrou est levé lorsque la transaction T1 est terminée, la transaction T2 pourra alors poser elle-même un verrou pour son usage.
- Notons qu'une transaction doit maintenir son verrou sur une donnée aussi longtemps qu'elle y a accès. De plus, il n'est pas toujours souhaitable qu'une transaction déverrouille la donnée qu'elle traite immédiatement après y avoir eu accès, puisqu'on n'est pas sûr de sa sérialisabilité.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Le verrouillage**
- Modes de verrouillage
- Le mode partagé: dans ce mode, une transaction peut lire un objet mais non l'écrire. C'est un verrou en lecture. Aucune transaction T ne peut mettre à jour la donnée E mais des transactions concurrentes T4, T3 ... peuvent l'accéder en lecture.
- Le mode exclusif: dans ce mode, une transaction peut lire et écrire l'objet. C'est un verrou en écriture. Seulement la transaction T peut mettre à jour la donnée E et tout autre accès est interdit.
- Le mode partagé est toujours compatible avec lui-même, mais incompatible avec le mode exclusif. On peut entretenir plusieurs modes partagés, à un instant donné, entre plusieurs transactions pour un objet donné, c'est à dire, plusieurs verrous en lecture sur un même objet.
- Une demande ultérieure de mode exclusif doit attendre la relaxation du mode partagé pour être prise en compte.



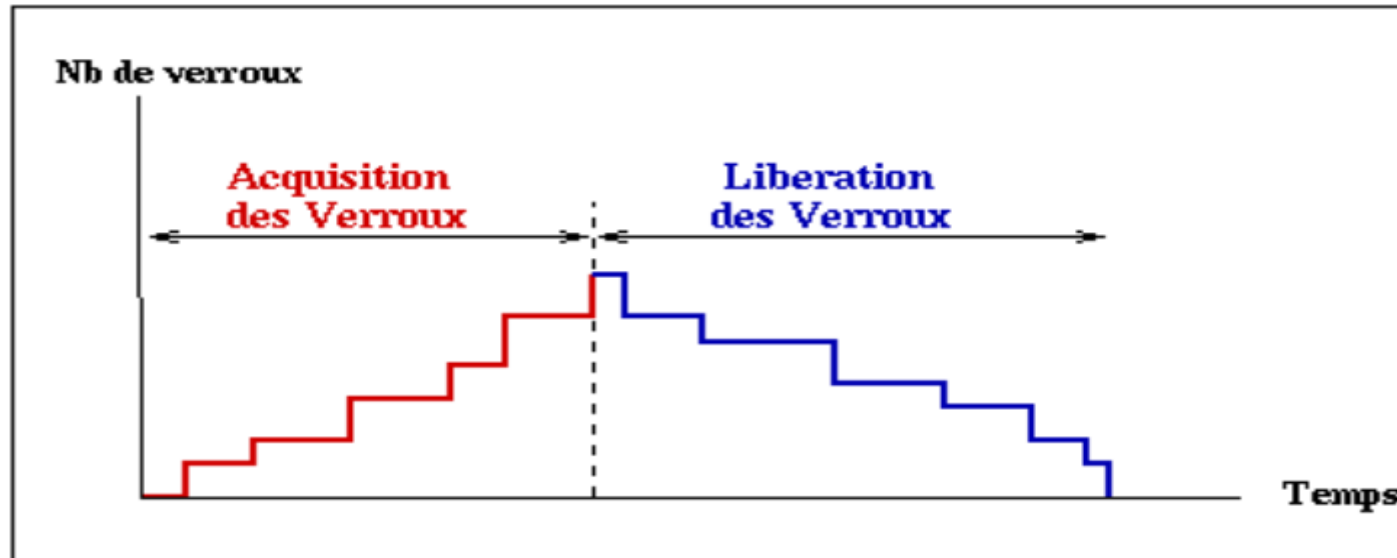
CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Le protocole de verrouillage deux-phases.**
- On a vu que le verrouillage des données d'une base doit être appliqué avec modération. Si, d'une part, on tente d'augmenter le rendement du traitement simultané des transactions en déverrouillant les données dès que possible, la base peut se retrouver en état inconsistant. Si d'autre part on ne déverrouille pas un article avant d'en verrouiller un autre, on crée des inter blocages.
- Nous imposerons donc aux transactions de respecter un ensemble de règles appelées protocole de verrouillage qui organise le verrouillage et le déverrouillage des données concernées par les transactions.
- Le protocole de verrouillage le plus important est le protocole deux-phases. Si toutes les transactions utilisent ce protocole, alors tous les schémas parallèles qu'on peut former seront sérialisables.
- Ce protocole implique que chacune des transactions de la séquence émette ses demandes de verrouillage et de déverrouillage en deux phases :



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Verrouillage croissant** : Une transaction peut obtenir des verrouillages, mais pas de nouveau déverrouillage. On pose tous les verrous nécessaires pendant cette phase.
- **Verrouillage décroissant** : Une transaction peut obtenir des déverrouillages, mais pas de nouveau verrouillage.



- Initialement, une transaction donnée est en phase de verrouillage croissant. Lorsqu'elle libère un verrou, elle entre en phase de décroissance et aucun verrou ne peut plus être imposé.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Algorithme**
 - **VERROUILLAGE EXCLUSIF**
 - Un ordre de priorité pour les transactions est choisi
 - Chaque quantum de temps, l'ordonnanceur passe la main à la première transaction dans la liste de priorité choisi.
 - Une transaction T_i demande un objet Ok.
 - S'il n'y a pas un verrou sur l'objet Ok , on verrouille Ok.
 - S'il y a un verrou sur l'objet Ok, alors la transaction T_i est mise en attente et le processeur est attribué à la transaction suivante dans la liste de priorité.

Lorsqu'une transaction est validée, on lui affecte la priorité la plus basse.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Avantages et inconvénients**
 - Un mécanisme de verrouillage permet à une transaction de se réserver l'usage exclusif d'une donnée aussi longtemps que c'est nécessaire.
 - Un inconvénient du verrouillage est son coût élevé lorsque les transactions font référence à de nombreuses données de la base.
 - En contrepartie, le degré de parallélisme est maximum puisque seules sont verrouillées les données réellement manipulées par les transactions.
 - On doit aussi tenir en compte le problème de verrous mortels.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Estampillage**
 - Dans le paragraphe précédent, nous avons étudié la méthode de verrouillage qui, associée à un protocole deux phases (par exemple) garantit la sérialisabilité d'une exécution de transactions en parallèle.
 - Si une transaction demande un objet qui est bloqué, il doit attendre que l'objet se libère : L'ordre est fixé à l'exécution suivant l'ordre dans lequel les transactions successivement bloquent les objets, ie demandent les objets.
 - Ici, la méthode de l'estampillage est une autre approche du problème: L'ordre est fixé à priori suivant l'ordre d'apparition des transactions; notre but ici étant toujours de garantir la sérialisabilité d'une exécution.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Estampille d'une transaction.**
- Une estampille doit identifier de façon unique une transaction et doit être créée avant l'exécution de la transaction.
- Les estampilles sont gérées par le SGBD et sont assignées typiquement dans l'ordre chronologique de soumission des transactions au système.
- Ainsi, on peut considérer l'estampille d'une transaction comme la date de début de celle-ci.
- Le système doit gérer les estampilles de façon que chaque estampille soit unique.
- Pour ce faire, il y a deux méthodes :
- Utilisation de l'horloge système pour dater les transactions. Il faut alors vérifier l'unicité des estampilles.
- Utilisation d'un compteur qui s'incrémente à chaque fois que sa valeur est attribuée à une transaction: cette valeur est alors l'estampille de la transaction. Comme le compteur a une valeur maximale finie, il faut veiller à le remettre à zéro pendant une période où il n'y a pas de transactions à exécuter.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Estampille d'un objet.**
- L'estampille d'un objet correspond à l'estampille de la dernière transaction qui a accédé à cet objet.
- On distingue alors deux types d'estampille sur un objet :
 - **L'estampille en écriture** : correspond à l'estampille la plus élevée parmi celles des transactions ayant effectué avec succès une opération de lecture sur l'objet : la plus jeune de ces transactions.
 - **L'estampille en lecture** : correspond à l'estampille la plus élevée parmi celles des transactions ayant effectué avec succès une opération d'écriture sur l'objet : la plus jeune de ces transactions.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Principe de l'estampillage**
- L'idée de cette méthode est de mettre en oeuvre un protocole d'ordonnancement par estampilles : il s'assure que les opérations conflictuelles de lecture et d'écriture sont exécutées par ordre d'estampille.
- Supposons par exemple une exécution avec deux transactions en concurrence T1 et T2.
- $E(T1)$ est l'estampille de T1 et $E(T2)$ l'estampille de T2.
- Supposons que nous avons $E(T1)=1$ et $E(T2)=2$.
- Comme $E(T1)>E(T2)$, on sait que l'ordre de soumission des transaction au système est T1 puis T2.
- Le protocole d'estampillage doit faire respecter cet ordre T1 puis T2.
- Voyons les quelques cas simples qui peuvent se présenter avec les deux transactions précédentes.
- Dans ce cas-ci, T1 lit O avant que T2 ne fasse une opération d'écriture sur l'objet. T1 précède bien T2 : On respecte l'ordre des estampilles.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Principe de l'estampillage**
 - Dans ce cas-ci par contre, T2 fait une écriture sur O avant que T1 ne fasse d'opérations sur l'objet.
 - L'ordre des estampilles n'est pas respecté. Le protocole règle ce problème, en abandonnant T1: T1 est alors soumis à nouveau au système avec une nouvelle estampille.
 - L'ordre des estampilles sera alors : T2 puis T1.
 - Il est clair que le protocole pour les tous autres cas qui peuvent se présenter gardera la même logique: On vérifie par le jeu des estampilles si une opération conflictuelle (lecture et écriture par 2 transactions différentes ou écriture et écriture) s'opère dans l'ordre dicté par les estampilles.
 - Si c'est le cas, on abandonne la transaction la plus vieille (celle qui a l'estampille la plus faible): Cet algorithme privilégie les transactions les plus récentes.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Technique de gestion de la concurrence(suite)
- Estampillage(Le protocole d'ordonnancement)
 - Enonçons l'algorithme d'estampillage:

On note $E(T)$ l'estampille de la transaction T .

On note $E_Lecture(O)$ l'estampille en lecture de l'objet .

On note $E_Ecriture(O)$ l'estampille en écriture de l'objet .

Supposons que la transaction T veuille faire une lecture sur l'objet O :

- *Si $E(T) < E_ecriture(O)$: Cela signifie que T est plus vieille que la plus vieille transaction qui a lu O . On viole l'ordre imposé par les estampilles l'opération de lecture de T doit être refusée et T doit donc être abandonnée.*
- *Si $E(T) \geq E_lecture(O)$: Cela signifie que T est plus jeune que la plus vieille transaction qui a fait une écriture sur O : L'ordre imposé par les estampilles est respecté. L'opération de lecture de T est alors acceptée .*
- *L'estampille en lecture de O est égale alors à la plus grande des deux valeurs: $E(T)$ ou $E_lecture(O)$.*



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Technique de gestion de la concurrence(suite)
- Estampillage(Le protocole d'ordonnancement)

- Enonçons l'algorithme d'estampillage(suite):

Supposons que la transaction T veuille faire une écriture sur l'objet O :

- *Si $E(T) < E_lecture(O)$: Cela signifie que T est plus vieille que la plus vieille transaction qui a lu O . On viole l'ordre imposé par les estampilles l'opération d'écriture de T doit être refusée et T doit donc être abandonnée.*
- *Si $E(T) \geq E_lecture(O)$: Cela signifie que T est plus jeune que la plus vieille transaction qui a fait une écriture sur O . L'opération de lecture de T est alors acceptée.*
- *L'estampille en écriture de O est égale alors à $E(T)$.*



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Avantages et inconvénients de l'estampillage**
 - Il est clair que l'estampillage définit un protocole qui permet de décider laquelle de deux transactions qui veut accéder à un objet doit être éliminée.
 - Ce protocole garantit la sérialisabilité de l'exécution.
 - De plus, contrairement au verrouillage, on est exempt de problème de blocage puisqu'aucune transaction n'attend une autre transaction.
 - Cependant, l'estampillage présente deux inconvénients majeurs, essentiellement liés au processus d'abandon de transaction : le livelock et le cascading roolback.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Avantages et inconvénients de l'estampillage**
 - **Le livelock**: Il peut arriver qu'une même transaction soit abandonnée et répétée continuellement : ce phénomène est appelé **livelock**.
 - **L'abandon en cascade**: Il peut arriver des phénomènes d'abandon en cascade qui sont pénalisant en terme de temps de réalisation.
 - Quand une transaction **T** est abandonnée, il faut prévoir les conséquences sur la base de données et toute transaction **T'** ayant utilisée les données écrites par **T** doit être à son tour abandonnée. Ceci se répercute de la même façon, pour toutes les transactions **T''** qui ont utilisé des valeurs de données modifiées par **T'**. Et ainsi de suite...
 - L'abandon effectif d'une transaction peut alors être très long.
 - En conclusion, on peut ajouter que l'estampillage est bien moins utilisé dans les entreprises que le verrouillage sûrement à cause de sa mise en oeuvre qui n'est pas évidente : le protocole utilise des abandons de transactions qui sont des processus très contraignants. On retrouve



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Les méthodes hybrides: Verrouillage-Estampillage**
- On retrouve néanmoins le protocole de l'estampillage dans des techniques de contrôle de concurrence hybrides entre verrouillage et estampillage .
- **Principe**
 - Dans le verrouillage on a vu qu'un **interblocage** peut se produire. La **prévention** des interblocages consiste à supprimer l'une des conditions qui rend possible les situations de **verrou mortel**.
 - Deux approches classiques sont le **préordonnement des ressources** et le **préordonnement des transactions**.
 - La première paraît difficile à mettre en œuvre du fait du grand nombre de données dans une base.
 - La deuxième a donné deux méthodes connues sous les noms de **Préventive sans droit de préemption (Die-Wait)** et **Préventive avec droit de préemption (Wound-Wait)**.
 - Dans les deux méthodes, les estampilles de transactions sont utilisées pour définir un préordre des transactions.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Technique de gestion de la concurrence(suite)
- Les méthodes hybrides: Verrouillage-Estampillage
- Les algorithmes
 - Préventive sans droit de préemption ou Die-Wait.
 - L'approche **Die-Wait** fonctionne comme suit : quand une transaction **T_i** demande à verrouiller un objet qui est déjà verrouillé par une transaction **T_j**, **T_i** attend **T_j** seulement si **T_i** est **plus vieille** que **T_j** ($E(i) < E(j)$) et meurt sinon.
 - Quand une transaction meurt, elle est ensuite reprise avec la même estampille de transaction si bien que tôt ou tard, elle deviendra la plus vieille transaction active et ne mourra plus.
 - Règle mise en oeuvre en cas de conflit entre **T_i** et **T_j** ($i < j$) :
 - *Si **T_j** attend **T_i** alors **T_j** est abandonnée et recommencée avec la même estampille,*
 - *Si **T_i** attend **T_j** alors **T_i** reste en attente.*



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- Technique de gestion de la concurrence(suite)
- Les méthodes hybrides: Verrouillage-Estampillage
- Les algorithmes
 - Préventive avec préemption ou Wound-Wait.
 - L'approche **Wound-Wait** est une approche symétrique mais avec **préemption**.
 - Quand **T_i** demande à verrouiller une donnée, **T_i** attend **T_j** si **T_i** est plus jeune que **T_j** ($E(i) > E(j)$).
 - Si **T_i** est plus vieille que **T_j**, **T_i** tue **T_j**.
 - Règle mise en oeuvre en cas de conflit entre **T_i** et **T_j** ($i < j$) :
 - Si **T_i** attend **T_j** alors **T_j** est abandonnée, la ressource est cédée à **T_i**,
 - Si **T_j** attend **T_i** alors **T_j** reste en attente.



CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Avantages et inconvénients des méthodes préventives**
- Dans les deux règles, on annule la transaction la plus jeune des deux transactions qui pourraient conduire à un **deadlock**. Ces deux méthodes permettent d'éviter les deadlocks.
- Cependant, elles peuvent conduire à l'annulation de transactions bien que ces transactions ne conduiraient pas à un deadlock.
- Par ailleurs dans la règle **Wait-Die**, la transaction **T_j** peut être annulée et reprise plusieurs fois si une transaction plus âgée **T_i** maintient son accès sur l'objet dont a besoin **T_j**. Par contre, la transaction **T_i** peut rester en attente indéfinie si de nouvelles transactions plus jeunes arrivent et accèdent avant elle à l'objet dont elle a besoin. Une transaction âgée peut ainsi n'être jamais exécutée.

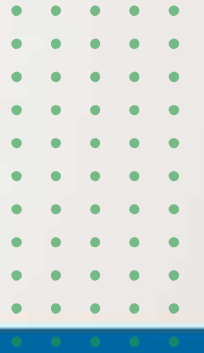


CHAPITRE 3 : GESTION DES TRANSACTIONS ET CONCURRENCE

- **Technique de gestion de la concurrence(suite)**
- **Avantages et inconvénients des méthodes préventives**
- Dans la règle Wound-Wait, en donnant un droit de préemption aux transactions plus âgées sur les transactions plus jeunes, on évite l'attente infinie d'une transaction. Tôt ou tard une transaction en attente est libérée ou devient elle même la plus âgée avec les droits de préemption sur les autres transactions.
- De plus, si on annule une transaction T_j , il est moins vraisemblable qu'elle sera à nouveau annulée par la suite. On a en effet changé l'ordre d'exécution, ce qui n'était pas le cas dans la règle Wait-Die. T_j à la reprise de son exécution risque surtout de se retrouver dans une situation d'attente de la fin de la transaction T_i qui l'a tuée.



10^{ans}
de proximité



MERCI